AWS Certified AI Practitioner (AIF-C01) — Complete Study Guide

All 5 Domains · In-Depth Notes · AWS Services Quick Reference

---

Domain 1: Fundamentals of AI and ML — 20%
Domain 2: Fundamentals of Generative AI — 24%
Domain 3: Applications of Foundation Models — 28%
Domain 4: Guidelines for Responsible AI — 14%
Domain 5: Security, Compliance, and Governance for AI Solutions — 14%

Compiled study notes · exam format: 65 questions (50 scored), 90 minutes, pass mark 700/1000

DOMAIN 1: FUNDAMENTALS OF AI AND ML

20% of scored exam content

Task Statement 1.1: Basic AI concepts and terminology

Nesting of fields: AI is the broadest field (machines performing human-like tasks). ML is a subset of AI (systems that learn patterns from data). Deep learning is a subset of ML (neural networks with multiple layers, good for images/speech/unstructured data). Generative AI is a subset of deep learning that creates new content using foundation models.

Key terminology

- **Model** = the trained artifact that predicts. **Algorithm** = the method used to train it.
- **Training vs. inference** — training is when a model learns from data; inference is when it makes predictions on new data.
- **Batch inference** — large volumes processed together, not time-sensitive (e.g. overnight fraud scoring).
- **Real-time inference** — instant response needed (e.g. live chatbot).
- **Bias** — systematic errors that skew output unfairly, often from unrepresentative data.
- **Overfitting** — memorizes training data, performs poorly on new data (low bias / high variance).
- **Underfitting** — too simple, performs poorly even on training data (high bias / low variance).
- **Labeled vs. unlabeled data** — labeled has the answer attached (supervised learning); unlabeled does not (unsupervised learning).
- **Structured vs. unstructured data** — tables/databases vs. text/images/audio.

Types of Machine Learning

Type	Data Used	How It Learns	Example
Supervised learning	Labeled data	Maps input → known output	Spam detection, price prediction
Unsupervised learning	Unlabeled data	Finds hidden patterns/groupings	Customer segmentation, anomaly detection
Reinforcement learning	Agent + environment + reward	Learns by trial & error	Game-playing AI, robotics

Within supervised learning: **Regression** predicts a continuous number; **Classification** predicts a category. Within unsupervised learning: **Clustering** groups similar items; **Anomaly detection** flags outliers.

Task Statement 1.2: Practical use cases for AI

When AI/ML adds value: automating repetitive tasks, scaling decisions across huge datasets, augmenting human judgment, personalization, detecting patterns humans would miss.

When AI/ML is NOT the right tool: when a guaranteed, deterministic outcome is required (e.g. calculating exact tax owed — use rule-based logic, not ML); when data is insufficient or low-quality; when the cost of building/maintaining a model outweighs the benefit.

AWS AI Services — one-line purpose (also feeds the master table at the end of this guide)

Service	Purpose
Amazon SageMaker	Build, train, and deploy custom ML models
Amazon Transcribe	Speech to text
Amazon Polly	Text to speech
Amazon Translate	Language translation
Amazon Comprehend	NLP — sentiment, entities, key phrases from text
Amazon Lex	Build conversational chatbots
Amazon Rekognition	Image and video analysis
Amazon Textract	Extract text/data from scanned documents
Amazon Personalize	Recommendation engines
Amazon Kendra	Intelligent enterprise search

Task Statement 1.3: The AI/ML development lifecycle

- 1. Business problem framing — define the goal and success metric
- 2. Data collection — gather relevant data
- 3. Data preprocessing / feature engineering — clean, label, transform data
- 4. Model selection — pick an appropriate algorithm/architecture
- 5. Model training — model learns from training data
- 6. Model evaluation — test on held-out data using metrics below
- 7. Deployment — put model into production for inference
- 8. Monitoring & maintenance — watch for model drift, retrain as needed

Model evaluation metrics

Metric	Meaning	Best For
Accuracy	% of total predictions correct	Balanced datasets only — misleading if imbalanced
Precision	Of predicted positives, how many were correct	False positives are costly (e.g. spam filter)
Recall	Of actual positives, how many were caught	False negatives are costly (e.g. disease detection, fraud)
F1 score	Harmonic mean of precision & recall	Need balance between the two

Exam trap: The exam rarely asks for a definition directly — it gives a scenario and expects you to apply the concept (e.g. 'a hospital wants to catch every cancer case even with false alarms' → optimize for recall).

DOMAIN 2: FUNDAMENTALS OF GENERATIVE AI

24% of scored exam content

Task Statement 2.1: Basic concepts of generative AI

Foundation Models (FMs) — very large models pre-trained on massive, broad datasets, then adapted to many downstream tasks via prompting or fine-tuning, instead of training a new model from scratch each time. This is the core shift GenAI represents over Domain 1's models.

Must-know terminology

- **Tokens** — basic units a model processes (roughly parts of words); all pricing/context limits are token-based.
- **Chunking** — breaking large documents into smaller pieces before feeding to a model (critical for RAG).
- **Embeddings** — numerical vector representations of text/images capturing meaning.
- **Vectors** — the arrays of numbers embeddings produce; stored in a vector database for similarity search.
- **Prompt engineering** — crafting inputs to get better outputs, without changing the model itself.
- **Transformer architecture** — the neural network architecture behind almost all modern LLMs; uses 'attention'.
- **LLM** — a foundation model specialized in understanding/generating text.
- **Multi-modal models** — models that take/produce more than one data type (text+image, text+audio).
- **Diffusion models** — used mainly for image generation; start from random noise, iteratively refine into an image.

Foundation model lifecycle: Data selection → Model selection → Pre-training → Fine-tuning → Evaluation → Deployment → Feedback.

Task Statement 2.2: Capabilities and limitations of GenAI for business problems

Where GenAI helps: adaptability (one model, many tasks), responsiveness, simple natural-language interface, content creation at scale.

Limitations (heavily tested):

- **Hallucinations** — confidently generating false/fabricated information. The #1 tested limitation.
- **Lack of interpretability** — hard to explain why an FM produced a specific output.
- **Non-determinism** — same prompt can give different outputs each time.
- **Bias** — inherited from training data, can be amplified in outputs.
- **Regulatory/compliance risk** — outputs may not automatically comply with regulations.
- **Cost at scale** — token-based pricing adds up with high-volume use.
- **Knowledge cutoff** — FMs only know training data up to a point in time (motivates RAG — see Domain 3).

Business decision framing: if a scenario needs exact, deterministic, explainable results (e.g. a regulated fee calculation), GenAI is the wrong tool — same logic as Domain 1's ML-vs-rules trap.

Task Statement 2.3: AWS infrastructure & technologies for building GenAI apps

Service / Tool	Purpose
Amazon Bedrock	Managed access to multiple FMs (Anthropic, Meta, Amazon, etc.) via one API, no infra to manage

Service / Tool	Purpose
Amazon SageMaker (AI)	Build, train, deploy custom ML/GenAI models
SageMaker JumpStart	Hub of pre-built, pre-trained FMs/solutions deployable quickly
Amazon Q	AWS's GenAI assistants (Q Business for enterprise Q&A;, Q Developer for coding)
Strands Agents	Framework for building AI agents (multi-step, tool-using workflows)
Amazon Bedrock AgentCore	Infrastructure/tooling within Bedrock for running agents at scale
Kiro	AWS's AI-powered IDE / agentic coding tool

DOMAIN 3: APPLICATIONS OF FOUNDATION MODELS

28% — heaviest domain of scored exam content

Task Statement 3.1: Design considerations for FM applications

Criteria for selecting a pre-trained foundation model

- **Cost** — larger/more capable models cost more per token.
- **Modality** — text-only vs. text+image vs. text+audio, etc.
- **Latency** — real-time chat needs low latency; batch tasks tolerate slower models.
- **Multi-lingual support** — check the model supports required languages.
- **Model size / complexity** — bigger isn't always better; smaller/cheaper is often enough.
- **Customization options** — can it be fine-tuned, or prompting only?
- **Input/output length (context window)** — long documents/conversations need a large context window.

Inference parameters

Parameter	Effect
Temperature (low)	More deterministic, focused, repeatable output — use for factual Q&A;, code
Temperature (high)	More random, creative, varied output — use for brainstorming, creative writing
Top-p (nucleus sampling)	Limits token choice to smallest set whose combined probability reaches p
Top-k	Limits next-token choice to top k most likely candidates
Max tokens	Caps how much a model reads/generates — controls cost & context window use

Trap: temperature/top-p/top-k control randomness/creativity, NOT factual accuracy. 'More consistent answers' → lower temperature, not a different model.

Retrieval-Augmented Generation (RAG) — most tested concept in the exam

Problem it solves: FMs only know training data (knowledge cutoff) and can hallucinate on private/proprietary/very recent information.

RAG flow: (1) Source docs are chunked → (2) each chunk is embedded into a vector → (3) vectors stored in a vector database → (4) user query is embedded the same way → (5) similarity search finds closest chunks → (6) retrieved chunks inserted into the prompt as context → (7) FM generates a response grounded in that context.

On AWS: **Amazon Bedrock Knowledge Bases** (managed RAG pipeline), **Amazon Titan Embeddings** (text→vector), vector stores such as **OpenSearch Service**, **Aurora/RDS with pgvector**.

RAG vs. fine-tuning: RAG is cheap and instantly updatable — use when the problem is missing/changing **knowledge**. Fine-tuning changes model weights — use when the problem is **behavior/skill/tone**, not facts.

Task Statement 3.2: Prompt engineering techniques

Technique	What It Is	Example
Zero-shot	No examples given	"Translate this to French: ..."
Single-shot	Exactly one example given	Show 1 formatted example, then ask
Few-shot	Several examples to set a pattern	3–5 example Q&A; pairs, then the real question
Chain-of-thought	Model reasons step-by-step before answering	Improves math/logic/multi-step reasoning
Prompt templates	Reusable structured formats with placeholders	"You are a support agent. Question: {q}..."

Risks of prompt engineering — easy to mix up

- **Prompt injection/hijacking** — hidden instructions override the system's original intent.
- **Prompt poisoning** — malicious content inserted into a prompt or a data source pulled into context.
- **Jailbreaking** — prompts specifically crafted to bypass safety guardrails.
- **Exposure** — sensitive info (system prompt, private data) unintentionally revealed.

Agents

Amazon Bedrock Agents extend an FM to take actions — calling APIs ('action groups'), querying a Knowledge Base, executing multi-step workflows — not just generating text. Key terms: **orchestration** (breaking a goal into steps), **tool usage**, **memory management**, **Model Context Protocol (MCP)**, **multi-agent systems**. If a scenario needs multi-step real-world actions across systems → it's an agent use case, not a simple prompt/RAG task.

Task Statement 3.3: Training and fine-tuning FMs

Stage	What Happens	Cost
Pre-training	Learns general patterns from massive unlabeled data	Extremely high (done once by provider)
Continuous pre-training	Further pre-training on domain-specific unlabeled data	High
Fine-tuning	Adapts a pre-trained FM using smaller labeled data	Moderate
Distillation	Smaller 'student' model mimics a larger 'teacher' model	Reduces inference cost

Fine-tuning methods: **Instruction tuning** (better at following directions generally), **Domain adaptation** (field-specific vocabulary — legal/medical/finance), **Transfer learning** (reusing a model trained on one task for a related task). **RLHF** (Reinforcement Learning from Human Feedback) — humans rank outputs → trains a reward model → aligns the model to human preferences.

The Customization Decision Ladder (cheapest → most expensive)

Rung	Trigger Phrase	Cost
1. Prompting / in-context learning	Wording/instructions need improvement	Cheapest

Rung	Trigger Phrase	Cost
2. RAG	Doesn't know current/private/changing data	Cheap
3. Fine-tuning	Needs permanent change in tone/format/domain skill	Moderate
4. Continuous pre-training	Needs deep, broad domain understanding first	Expensive
5. Pre-training from scratch	Needs a wholly new foundation model	Extremely expensive (rarely correct answer)

Golden rule: knowledge problem → RAG. Skill/behavior problem → fine-tuning.

Task Statement 3.4: Evaluating FM performance

- **Human evaluation** — people review outputs for quality/nuance (slower, catches what metrics miss).
- **Benchmark datasets** — standardized datasets with known answers for consistent automated scoring.

Metric	Full Name	Used For	Memory Hook
ROUGE	Recall-Oriented Understudy for Gisting Evaluation	Summarization	Recall + summaRization
BLEU	Bilingual Evaluation Understudy	Translation	BiLingual = transLation
BERTScore	—	Semantic similarity	Understands meaning, not just word overlap

Business value metrics: productivity/efficiency gains, ROI, conversion rate, average revenue per user, cross-domain performance, customer lifetime value — tie technical evaluation to business outcomes.

DOMAIN 4: GUIDELINES FOR RESPONSIBLE AI

14% of scored exam content

Task Statement 4.1: Explain the development of responsible AI systems

Core features of responsible AI

- **Fairness** — treats different groups/individuals equitably.
- **Bias** — systematic skew, usually from unrepresentative training data.
- **Inclusivity** — works well for diverse users, not just a majority group.
- **Robustness** — performs reliably even with unexpected/edge-case inputs.
- **Safety** — avoids producing harmful outputs or actions.
- **Veracity** — outputs are truthful and accurate (ties to hallucination).

Legal risks of working with GenAI

- Intellectual property infringement (model trained on copyrighted data)
- Biased model outputs → discrimination liability
- Loss of customer trust / reputational risk
- End user risk — harm caused directly to users
- Hallucinations — false info with compliance consequences

Dataset characteristics to check: **Inclusivity, Diversity, Curated data sources, Balanced datasets** (imbalance → biased model, same trap as Domain 1's fraud accuracy example).

AWS tools for detecting/monitoring bias, trustworthiness & truthfulness

Tool	Purpose
Amazon SageMaker Clarify	Detects bias in datasets/models (pre- & post-training) + explainability (feature importance)
SageMaker Model Monitor	Continuously monitors deployed models for data quality, bias drift, performance decay
Amazon Augmented AI (A2I)	Inserts human review into ML predictions (human-in-the-loop)
Amazon Bedrock Guardrails	Filters harmful content, blocks topics, enforces responsible behavior in GenAI apps

Responsible model-selection practices also include environmental considerations/sustainability — choosing an appropriately-sized model, not always the biggest.

Task Statement 4.2: Transparent and explainable models

- **Transparent model** — you can see/understand how it works internally (e.g. decision tree, linear regression).
- **Explainable model** — internals may be complex (deep neural net), but tools can explain a specific output after the fact.
- **White box** = transparent/interpretable but often lower performance. **Black box** = opaque/complex (deep learning, LLMs), higher performance, harder to explain.

Tool	Purpose
Amazon SageMaker Model Cards	Documentation of a model's intended use, training data, performance, limitations, risks
SageMaker Clarify	Feature importance — which inputs drove a specific prediction
Amazon Bedrock Model Evaluations	Evaluate/compare FMs on quality and responsibility metrics before choosing one
Open source models/data/licensing	Generally more transparent — training data & architecture can be inspected

Key tradeoff: **Interpretability vs. performance.** Simple/transparent models are easy to explain but often less accurate on complex tasks; complex models perform better but are harder to interpret. 'Must explain every decision to regulators' → favors a simpler, more transparent model.

Human-centered design for explainable AI: user feedback mechanisms; AI decision transparency (clearly communicating AI involvement and reasoning, especially for high-stakes decisions like loans or medical triage).

DOMAIN 5: SECURITY, COMPLIANCE, AND GOVERNANCE FOR AI SOLUTIONS

14% of scored exam content

Task Statement 5.1: Explain methods to secure AI systems

Concept / Service	Purpose
IAM roles, policies, permissions	Control who/what can access AI resources and what they can do (least privilege)
Encryption (at rest & in transit)	Protects stored data and data moving over the network; keys managed via AWS KMS
Amazon Macie	Discovers & protects sensitive data (PII) in S3 using ML classification
AWS PrivateLink	Private access to AWS services from your VPC, avoiding the public internet
AWS Shared Responsibility Model	AWS secures the cloud infrastructure; customer secures data/access/config in the cloud

Source citation & data origin

- **Data lineage** — tracking where data came from and how it's been transformed.
- **Data cataloging** — organizing/indexing datasets so origin is discoverable.
- **SageMaker Model Cards** — also documents training data sources here (reappears from Domain 4).

Secure data engineering best practices: assessing data quality; privacy-enhancing technologies (PETs) e.g. anonymization/differential privacy; data access control; data integrity.

AI-specific security & privacy considerations: application security, threat detection, vulnerability management, infrastructure protection, **prompt injection** (reappears from Domain 3, now framed as a security risk to defend against), encryption at rest and in transit.

Task Statement 5.2: Governance and compliance regulations for AI systems

Service	Purpose
AWS Config	Tracks resource configuration changes over time; checks compliance against rules
Amazon Inspector	Scans workloads for software vulnerabilities & unintended network exposure
AWS Audit Manager	Continuously collects evidence to assess compliance with frameworks
AWS Artifact	On-demand access to AWS's own compliance reports (ISO, SOC, etc.)
AWS CloudTrail	Logs every API call/user action — the 'who did what, when' service

Service	Purpose
AWS Trusted Advisor	Best-practice recommendations across cost, performance, security, fault tolerance

Quick differentiator: Config = configuration compliance over time · CloudTrail = who did what action · Audit Manager = evidence for an audit · Artifact = AWS's own compliance certificates · Inspector = vulnerability scanning · Trusted Advisor = general best-practice recommendations.

Data governance strategies: data lifecycles, logging, residency (geographic/legal data boundaries), monitoring & observation, retention.

Governance protocols: policies; review cadence/strategies; the **Generative AI Security Scoping Matrix** (AWS framework categorizing GenAI use cases by scope to determine right controls); transparency standards; team training requirements.

AWS Services Quick Reference — All Domains

Every AWS service/tool referenced across the exam guide, grouped by category, with a one-line purpose.

Generative AI & Model Access

Service	What It Does
Amazon Bedrock	Managed access to multiple foundation models via one API, no infrastructure to manage
Amazon Bedrock Knowledge Bases	Fully managed RAG — handles chunking, embedding, retrieval automatically
Amazon Bedrock Agents	Orchestrates multi-step tasks; lets an FM call tools/APIs ('action groups')
Amazon Bedrock AgentCore	Infrastructure for building/running agents at scale within Bedrock
Amazon Bedrock Guardrails	Filters harmful content, blocks topics, enforces responsible AI behavior
Amazon Bedrock Model Evaluations	Evaluate/compare foundation models on quality & responsibility metrics
Amazon Titan Embeddings	Converts text into vector embeddings for semantic search / RAG
SageMaker JumpStart	Hub of pre-built, pre-trained foundation models for quick deployment
Amazon Q	GenAI assistants — Q Business (enterprise data Q&A;), Q Developer (coding)
Strands Agents	Framework for building AI agents (multi-step, tool-using workflows)
Kiro	AWS's AI-powered IDE / agentic coding tool

Machine Learning Platform

Service	What It Does
Amazon SageMaker (AI)	Build, train, and deploy custom ML/GenAI models end to end
SageMaker Model Cards	Documentation of a model's intended use, training data, performance, limitations
SageMaker Clarify	Detects bias in data/models pre- and post-training; explains feature importance
SageMaker Model Monitor	Continuously monitors deployed models for drift, bias, and quality decay
Amazon Augmented AI (A2I)	Adds human review/oversight into ML prediction workflows

Applied AI Services (task-specific, pre-built)

Service	What It Does
Amazon Transcribe	Speech to text
Amazon Polly	Text to speech
Amazon Translate	Language translation
Amazon Comprehend	NLP — sentiment analysis, entity & key phrase extraction from text
Amazon Lex	Build conversational chatbots / voice interfaces
Amazon Rekognition	Image and video analysis (objects, faces, scenes)
Amazon Textract	Extract text, forms, and tables from scanned documents
Amazon Personalize	Build recommendation engines
Amazon Kendra	Intelligent enterprise search

Security

Service	What It Does
IAM (roles, policies, permissions)	Controls who/what can access resources and what actions they can perform
AWS KMS	Manages encryption keys for data at rest and in transit
Amazon Macie	Discovers and protects sensitive data (PII) in S3 using ML
AWS PrivateLink	Private connectivity to AWS services without traversing the public internet
AWS Shared Responsibility Model	Framework: AWS secures the cloud; customer secures what they put in the cloud

Governance & Compliance

Service	What It Does
AWS Config	Tracks resource configuration changes over time; checks compliance rules
Amazon Inspector	Scans workloads for vulnerabilities and unintended network exposure
AWS Audit Manager	Continuously collects evidence to assess compliance with frameworks
AWS Artifact	On-demand access to AWS's own compliance reports and certifications
AWS CloudTrail	Logs every API call and user action across your AWS account
AWS Trusted Advisor	Best-practice recommendations for cost, performance, security, fault tolerance

Tip: For every service above, be able to state its one-line purpose in under 5 seconds — the exam tests service-to-use-case matching far more than deep configuration knowledge.

MISTAKE REVIEW — REINFORCEMENT NOTES

Based on 12 missed practice questions · recurring traps to close out before exam day

1. Amazon Bedrock's feature family — these get confused constantly

Bedrock has multiple similarly-named features that each solve a completely different problem. Know which one answers which question:

Feature	Solves This Problem
Model access (request access)	Unlocking a Bedrock-provided BASE model so your account can call it
Provisioned Throughput	Required to run YOUR OWN custom/fine-tuned model in Bedrock — also used to guarantee dedicated capacity for a base model
Bedrock Knowledge Bases	RAG — indexing & retrieving documents to ground responses
Bedrock Guardrails	Filtering harmful/off-topic content for safety
Bedrock Agents / AgentCore	Orchestrating multi-step tasks and tool/API calls
Bedrock Prompt Management	Versioning prompts, comparing variants, rolling back if quality drops
Bedrock Model Evaluation	Scoring and comparing model quality/performance

Golden rule: scenario says "we trained/fine-tuned our OWN model and want to use it in Bedrock" → **Provisioned Throughput** (not SageMaker endpoints, not SageMaker Model Registry, not "model access," which is only for AWS-provided base models). Scenario says "track/compare/revert prompt versions" → **Prompt Management**, not Knowledge Bases or Guardrails.

2. Generative model architectures beyond transformers/diffusion

Model Type	Generative?	What It's For
GAN (Generative Adversarial Network)	Yes	A generator + a discriminator compete; produces general-purpose synthetic data (images, tabular, etc.)
WaveNet	Yes	Specialized generative model for raw AUDIO/speech synthesis only
ResNet (Residual Neural Network)	No	A deep CNN architecture for image recognition/classification, not generation
XGBoost	No	Gradient-boosting algorithm for supervised classification/regression

Trap: "needs to generate synthetic data" does not automatically mean GAN — check the modality first. Audio/speech specifically → WaveNet. General/mixed data → GAN. And don't be fooled by ResNet or XGBoost appearing as distractors — neither one generates anything.

3. Transformer self-attention — the exact mechanism, not just the name

- **Self-attention** weighs the importance of each token relative to every other token in the input, letting the model capture contextual relationships across an entire sequence — and it does this in parallel, not step-by-step.
- **Contrast with CNNs** — convolutional layers apply filters across input to capture local patterns (spatial, image-focused).
- **Contrast with RNNs** — process a sequence one element at a time in repeated cycles; slower and struggle with long-range dependencies.
- Trap: transformers are NOT limited to text — multimodal transformers exist (text+image, etc.). Don't pick an option that claims otherwise just because LLMs are the most familiar example.

4. Vector database options on AWS — know what is NOT a vector database

Service	Vector Search?	What It Actually Is
Amazon OpenSearch Service	Yes	Supports vector storage & similarity search over embeddings — the go-to answer for RAG
Amazon Aurora / RDS (pgvector)	Yes	Relational DB with a vector extension for embeddings
Amazon DynamoDB	No	Key-value / document database — no native vector search
Amazon ElastiCache	No	In-memory cache for low-latency lookups, not similarity search
Amazon Redshift	No	Data warehouse for analytics, not vector similarity search

5. RAG cost-effectiveness ranking — reinforced with a concrete scenario

For "ground an LLM's answers in internal documents, at the lowest cost," rank the options from cheapest to most expensive:

- **RAG** — retrieves only the relevant chunks per query; cheapest and needs no retraining. Correct answer in almost every version of this scenario.
- **Pasting all documents into every prompt** — NOT free. This balloons token costs on every single call and can exceed the context window. A common wrong-but-tempting distractor.
- **Fine-tuning on the documents** — real training cost, and must be repeated every time the documents change.
- **Pre-training a new model from scratch** — by far the most expensive; essentially never correct in a business scenario.

6. Keeping fine-tuning/data private and in-Region — the right architecture

When a scenario needs data privacy and Region residency during fine-tuning, the cost-effective managed-service answer is: use the **Amazon Bedrock API** (keeps everything inside the managed service, in your chosen Region) routed through **AWS PrivateLink with a VPC** (keeps traffic off the public internet).

Trap: hosting the model on premises with **AWS Outposts**, or hosting the Bedrock API on premises, both add real hardware/operational cost and are not how the managed Bedrock service is actually consumed — these are only correct when a scenario explicitly demands on-premises infrastructure, not just "privacy."

7. Agent-building tools — don't confuse with applied AI services

Strands Agents — an open-source Python SDK from AWS for building custom AI agents: supports tool use, multi-agent orchestration, and integration with multiple model providers (not locked to one vendor's models). Distinguish from

Amazon Bedrock Agents/AgentCore (the managed, in-Bedrock orchestration option). Neither Amazon Comprehend (NLP tasks like sentiment analysis), Amazon Personalize (recommendations), nor Amazon Lex (conversational chatbots) build custom multi-agent systems — a common distractor pattern.

8. Quick-hit reinforcements

- **SageMaker Clarify** — the exact tested phrase is "feature attribution" / explaining how input features influence a model's behavior. SageMaker Ground Truth labels data; SageMaker Feature Store manages features; SageMaker Canvas builds no-code models — none of these explain feature influence.
- **Amazon Q Business** — the trigger phrase "ask questions about internal company data" points here every time. Amazon Lex is a chatbot-building framework, not a ready-made enterprise data assistant — don't pick it just because it also involves conversation.
- **Fine-tuning** in the FM lifecycle = supervised training on a small LABELED dataset for a specific task. Pre-training/continuous pre-training use large, mostly unlabeled data; deployment is putting the finished model into production.

These 8 points came directly from your missed questions — review this page last, right before the exam, since it targets your actual gaps rather than general content.